Reflection and Traceability Report on Software Engineering

Team #1, Sanskrit Ciphers
Omar El Aref
Dylan Garner
Muhammad Umar Khan
Aswin Kuganesan
Yousef Shahin

This report is our record of how the project changed after feedback, and why it changed. We wrote it to show the connection between the comments we received, the revisions we made, and the lessons we learned while finishing the project.

1 Changes in Response to Feedback

Across the project, the most useful feedback was not about one single deliverable but about consistency. We were repeatedly reminded to keep the documents aligned with each other, to make the revision history clearer, and to make the technical story easier to follow for someone who is not already deep in the implementation.

The feedback also pushed us to be more honest about the current state of the system. In earlier versions we sometimes wrote as if the design were more mature than it actually was. The revisions forced us to be more precise: if a module was planned but not implemented, it should not appear as if it were part of the final architecture. If a model was still brittle, we needed to say so directly.

1.1 SRS and Hazard Analysis

oprule extbfFeedback	**Source**	**Response**
The SRS had drifted away from the current implementation state.	TA / document review	We rebuilt the PDF from the latest source, checked the requirements against the codebase and the design, and tightened wording so the SRS matches the current project scope.
The terminology across the requirements documents was not fully consistent.	Internal review	We standardised module names, exception names, and the glossary so that the SRS, MIS, and hazard analysis describe the same system using the same vocabulary.
The hazard analysis should connect more directly to real project risks.	Team review	We updated the hazards to focus on the actual failure points in our system: image ingestion, preprocessing, ML inference, storage, and user workflow issues.

This round of revisions made the requirements package much easier to read and much easier to compare against the rest of the documentation. It also made

the project feel more grounded, because the risks and assumptions are now tied to the system we actually built instead of a more abstract idea of the system.

1.2 Design and Design Documentation

oprule extbfFeedback	**Source**	**Response**
The design document described modules that were no longer part of the final architecture.	Internal consistency review	We removed unsupported references, including the unused `ML.TextSim` module, and brought the decomposition in line with the actual code structure.
Some interface descriptions were still too vague for grading or implementation.	TA / peer review	We added explicit `pre:` and `post:` style semantics and made the exception behavior more concrete for the major routines.
The design language did not always match the implementation language.	Team review	We renamed or rephrased several items so the service layer, data layer, UI, and ML sections use the same terminology throughout the documentation.

This was one of the most important revisions because it improved the connection between the architecture and the implementation. The document reads more like a single coherent design now, rather than a collection of partially updated sections.

1.3 VnV Plan and Report

oprule extbfFeedback	**Source**	**Response**
The evaluation section needed stronger evidence and better interpretation.	TA feedback on ML work	We added more task-specific metrics, highlighted where the strongest and weakest models were, and expanded the analysis of failures instead of relying only on summary numbers.
The report needed to be more critical of classical computer vision components.	Internal review	We added clearer limitations for edge detection and scale detection, especially where heuristics are sensitive to thresholds or fragile on damaged images.
The documents should be reproducible from source.	Submission review	We verified the report structure, kept the build files with the documents, and confirmed that the PDFs can be produced from the checked-in LaTeX sources.

The V&V revision was important because it changed the tone of the report. The final version does not just say that the models work; it shows where they work, where they fail, and what we learned from that.

2 Extras

We treated the extra deliverables as part of the project, not as side work. They helped us think about the system from more than one angle.

- **User documentation:** Writing user-facing material forced us to explain the system in plain language, which helped us catch jargon and workflow gaps.
- **Code walkthroughs:** Walkthroughs made it easier to share understanding across the team and made design decisions easier to justify.
- **Usability work:** We kept adjusting the upload, search, and workspace flows because we wanted the system to be understandable to someone outside the team.
- **Supporting analyses:** The supporting reports around cost, performance, green computing, wireframes, and related topics helped us think about the project as a complete product rather than just a set of models.

The biggest benefit of the extras was that they forced us to step back from the model-building mindset and look at the project as something a real user would have to interact with.

3 Design Iteration (LO11 (PrototypeIterate))

The design changed a lot over the course of the project, but the biggest change was not a single feature. It was the way we thought about the system.

At the start, we mostly described the project as a fragment matching tool. That was the right broad idea, but it was too vague to build on directly. As we worked through the implementation, we realised that the problem had to be split into smaller stages: preprocessing, segmentation, line detection, script classification, edge detection, circle detection, scale detection, and finally the service and UI layers that expose the results.

We made that shift because the client needs a tool that is useful on messy, real-world manuscript images, not on idealized examples. The fragments are not uniform. Some are damaged, some have rulers, some have annotations, and some have features that are easier to detect with classical vision than with a learned model. Once we accepted that, the modular design made more sense. Each component could be tuned for the type of signal it handles best.

The iteration also happened at the documentation level. Early versions of the documents described ideas that sounded complete, but the implementation forced us to be more careful. That helped us improve the actual design because we had to write down what was really true rather than what we hoped would be true.

4 Design Decisions (LO12)

Several choices were driven by constraints rather than preference.

extbfLimitations: We did not have a large balanced dataset for every task, so we chose approaches that could still perform reasonably well in small-data settings. That is why transfer learning, weighted sampling, and a mix of learned and heuristic methods appear throughout the project.

extbfAssumptions: We assumed the images would contain at least some usable visual signal and that the metadata would be kept consistent across the pipeline. Those assumptions made it necessary to add validation and error handling so the system fails clearly instead of producing misleading output.

extbfConstraints: We had to keep the project maintainable and understandable within a course timeline. That influenced the decision to keep the architecture modular and the documentation explicit. We did not want to hide complexity; we wanted to structure it so that it could be discussed, tested, and revised.

In the end, the design is intentionally conservative. We preferred explainable choices that fit the data we have over more complicated choices that would be

harder to justify or maintain.

5 Economic Considerations (LO23)

We do not think of this as a consumer product, so the economics are different from a normal app or SaaS project. The most realistic users are scholars, graduate students, librarians, and digital humanities researchers. The value is in saving expert time and making fragment analysis more scalable.

If we ever tried to productize it, the expensive parts would be data preparation, annotation, model training, and ongoing support. The dataset is specialized, so the real cost is not just software development but the expert time required to label and validate the outputs. Because of that, open access and academic collaboration make more sense than a conventional sales model.

To attract users, we would rely on demonstrations, documentation, reproducible results, and collaboration with research groups rather than advertising. The potential user base is limited in size, but that is normal for a tool aimed at a very specific research problem.

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

Overall, we stayed close to the development plan. We used GitHub issues and branches, communicated regularly, and split tasks by subsystem so people could work in parallel without constantly blocking one another. We also used the tools we said we would use: LaTeX for the documentation, Python for the ML work, and web technologies for the user-facing pieces.

The main difference was that the documentation effort grew larger than expected. Once the implementation settled, we had to spend extra time making sure every document reflected the same naming, scope, and level of detail. That was not a bad surprise, but it did show that the docs needed more attention than we first assumed.

6.2 What Went Well?

The team stayed communicative and responsive. We generally knew who owned each piece of work, and that made it easier to move quickly. Smaller tasks were easier to review and easier to merge. The modular codebase also helped, because people could focus on one part of the system without needing every other part to be finished first.

6.3 What Went Wrong?

The biggest problem was synchronization. When one part of the system changed, we often had to update several documents at once so that the terminology stayed consistent. We also underestimated how much time would go into formatting, cleanup, and build issues near the end. That work matters, but it can easily take time away from the actual technical writing if it is left too late.

6.4 What Would you Do Differently Next Time?

Next time, we would define the document structure and terminology earlier and keep them synchronized more aggressively. We would also make submission-ready build checks part of the regular workflow instead of treating them as a final cleanup task. That would reduce the last-minute pressure and make the final revision smoother.

7 Ethics and Equity (LO18)

We tried to design the project so that it supports human judgment rather than replacing it. That matters in a cultural heritage context, where the cost of overclaiming is high. The system should help scholars inspect, compare, and organize fragments; it should not pretend to make final historical judgments on its own.

We also thought about accessibility and clarity. The interface and documents need to work for people with very different technical backgrounds. That is why we favored direct labels, obvious workflows, and transparent outputs. If the user has to guess what the model did, the system is not helping enough.

We also tried to keep the presentation honest. For a project like this, it is important not to turn experimental results into certainty. Equity is not only about who can use the tool, but also about whether the tool represents its limitations fairly.

8 Reflection on Capstone

This capstone taught us that the hard part of a project is not always the core algorithm. A lot of the work was about coordination, revision, and explaining the system clearly enough that someone else could understand it without being in the room when the decisions were made.

8.1 Which Courses Were Relevant

The most relevant courses were software design, databases, machine learning, web development, software testing, and data structures and algorithms. Those courses gave us the technical base for the project, but the capstone also depended

on writing and documentation skills because a big part of the deliverable was explaining how the system works.

8.2 Knowledge/Skills Outside of Courses

We also had to learn several things that were not taught directly in class. These included working with OCR tools, training and evaluating YOLO-based pipelines, handling dataset labeling decisions, debugging model training issues, and producing large LaTeX documents that stay internally consistent across many sections.

Just as important, we had to learn how to make good engineering decisions when we did not have perfect data. That meant choosing practical methods, documenting the trade-offs, and being honest about uncertainty. That is probably the most useful lesson we will carry forward from the project.